

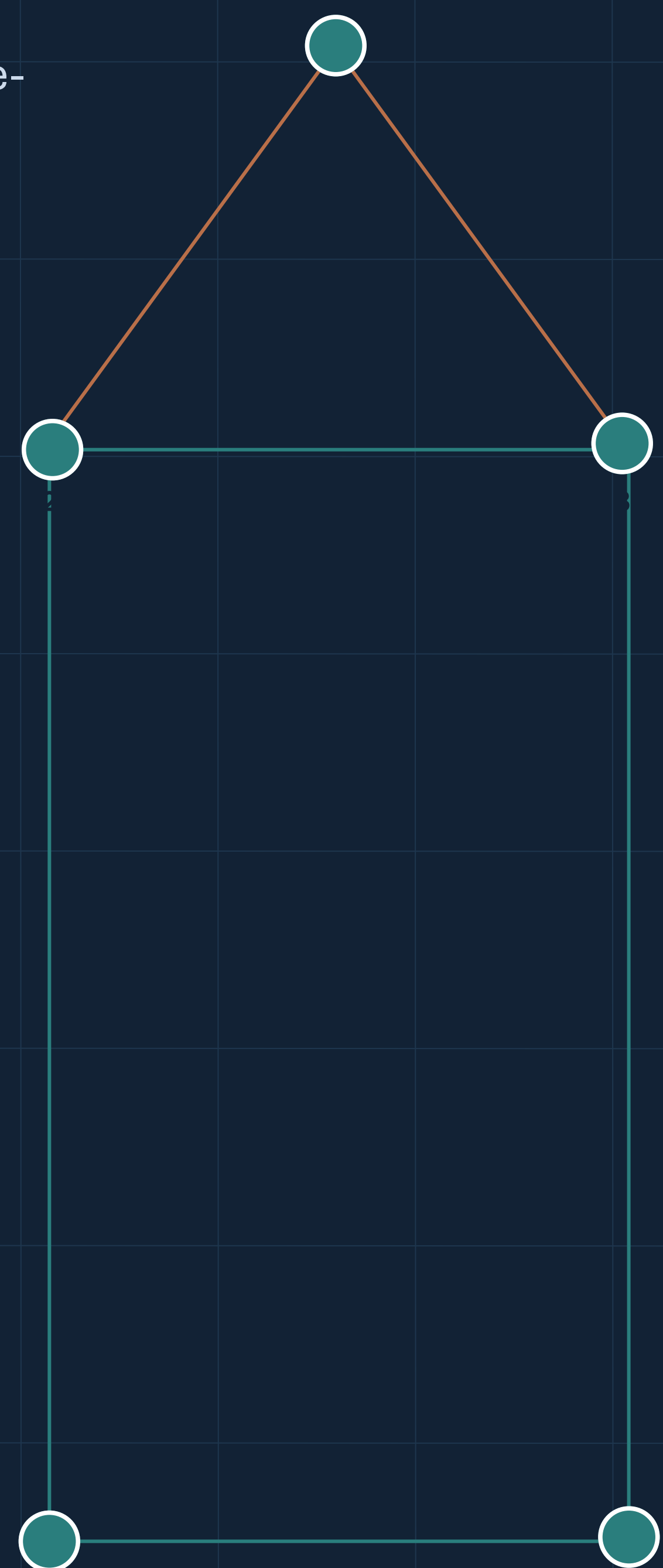
Computational Structural Analysis Using the Direct Stiffness Method

A python based direct strength method workflow for elastic 2D plane-frame analysis — from Excel model definition to matrix assembly, solution and engineering results.

DSM

2D FRAME

PYTHON + NUMPY



PREPARED BY

Sejal

Structural Analysis • Finite Element Method

What has been built?

A generalized Python application that converts a structural model into a solved 2D frame response.

THE PROJECT

This project implements the Direct Stiffness Method for elastic 2D plane frames and wraps the numerical engine in a graphical application. Geometry, member properties, supports and loads are supplied through Excel; the solver assembles and solves the global stiffness system and returns engineering results.

INPUT

Excel-defined structural model

ENGINE

DSM matrix formulation

OUTPUT

Nodal displacements, reactions and member forces

SCOPE

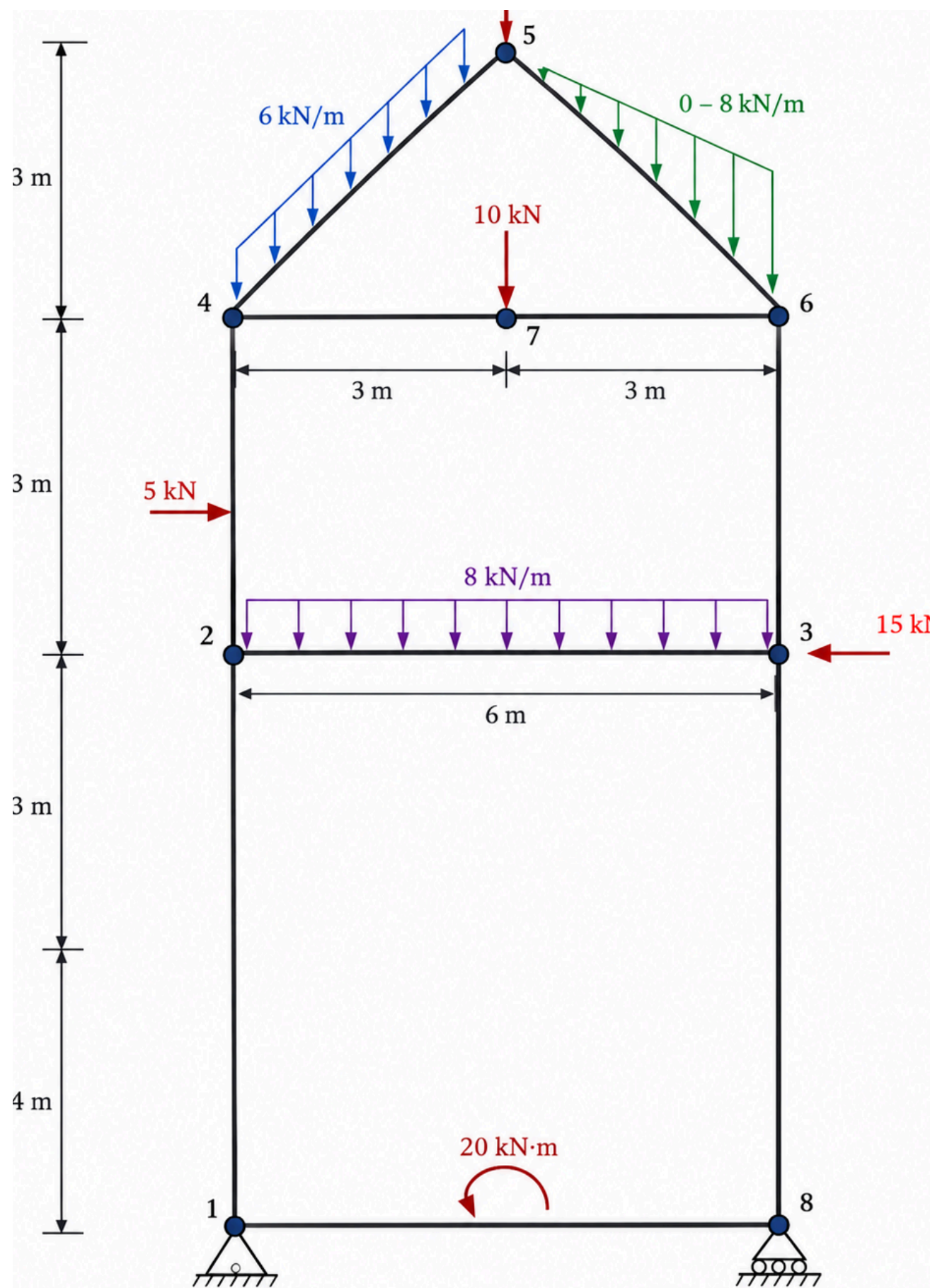
- Linear-elastic static analysis of 2D plane frames.
- Three DOFs per node: U_x , U_y and R_z .
- Horizontal, vertical and inclined members through coordinate transformation.
- Nodal actions plus member distributed-loading representation.
- Interactive inspection and export of computed results.

Design philosophy: make the mechanics traceable, repeatable and usable — not hidden behind a black-box calculation.

Why the Direct Stiffness Method?

DSM gives one systematic matrix framework for assembling many frame members into one equilibrium problem.

THE ENGINEERING PROBLEM



As geometry, supports and loading become more complex, writing separate equilibrium equations becomes cumbersome.

- Many members
- Inclined members
- Multiple restraints
- Indeterminate systems

WHAT DSM PROVIDES

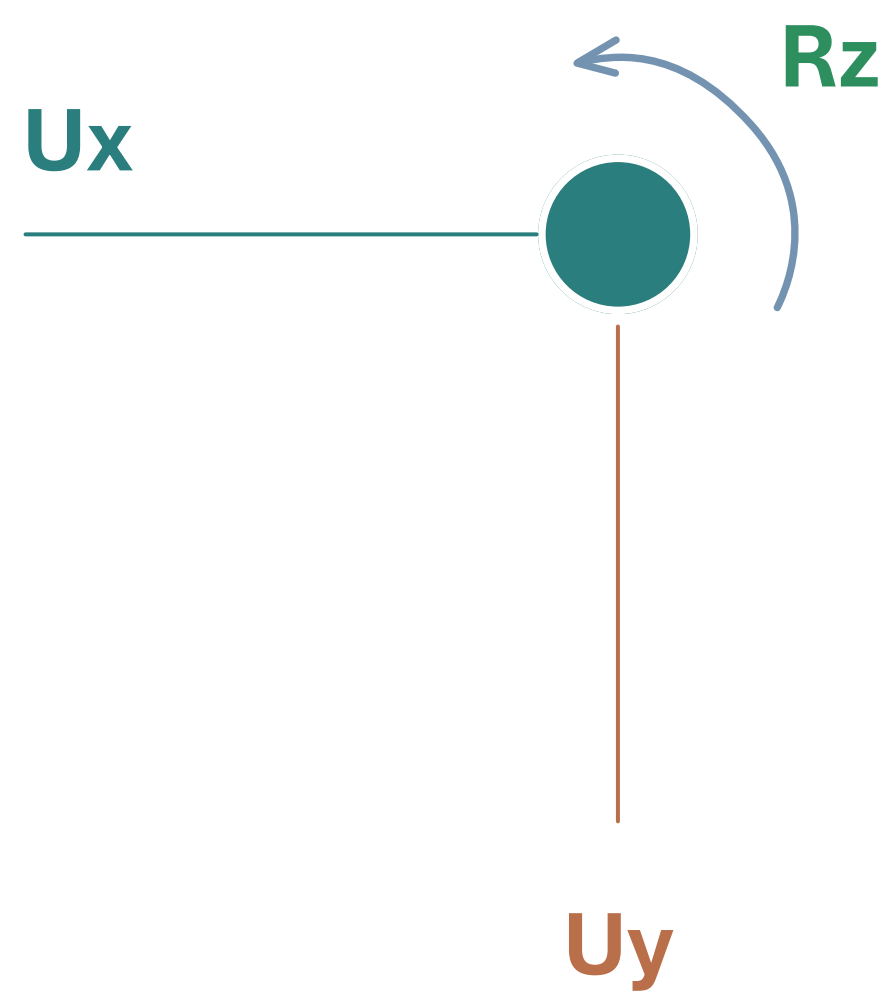
- 01 Element formulation**
A repeatable stiffness matrix for each member.
- 02 Transformation**
The same local formulation works for arbitrary orientation.
- 03 Global assembly**
All member contributions enter one global K matrix.
- 04 Boundary conditions**
Supports become constraints on global DOFs.
- 05 Post-processing**
Displacements lead to reactions and member forces.

The project uses DSM as the computational backbone and exposes the resulting workflow through a dedicated interface.

Three Degrees of Freedom at Every Node

The 2D frame formulation tracks two translations and one rotation at each node.

NODE-LEVEL DOFs



GLOBAL DOF ORDER

[Ux Uy Rz]

For n nodes:
Total DOF = 3n

IN YOUR IMPLEMENTATION

```
total_dofs = n_nodes * 3
```

```
get_dof(node, 0) → Ux
```

```
get_dof(node, 1) → Uy
```

```
get_dof(node, 2) → Rz
```

The DOF map determines global matrix locations.

Support flags classify DOFs as free or fixed.

The same indexing is reused for member recovery.

Physical structure → nodal DOFs → algebraic unknowns

The 6×6 Local Stiffness Matrix

Each frame member combines axial and flexural behavior in a single local stiffness matrix.

[k]_e — LOCAL STIFFNESS

EA/L	0	0	−EA/L	0	0
0	12EI/L ³	6EI/L ²	0	−12EI/L ³	6EI/L ²
0	6EI/L ²	4EI/L	0	−6EI/L ²	2EI/L
−EA/L	0	0	EA/L	0	0
0	−12EI/L ³	−6EI/L ²	0	12EI/L ³	−6EI/L ²
0	6EI/L ²	2EI/L	0	−6EI/L ²	4EI/L

TERM INTERPRETATION

EA/L	Axial stiffness
EI/L ³	Bending stiffness
EI/L ²	Coupling terms
EI/L	Rotational stiffness

DIRECTLY USED IN THE SOLVER

```
k_local = local_stiffness(E, A, I, L)
```

E, A and I come from the member definition; L comes from the node coordinates.

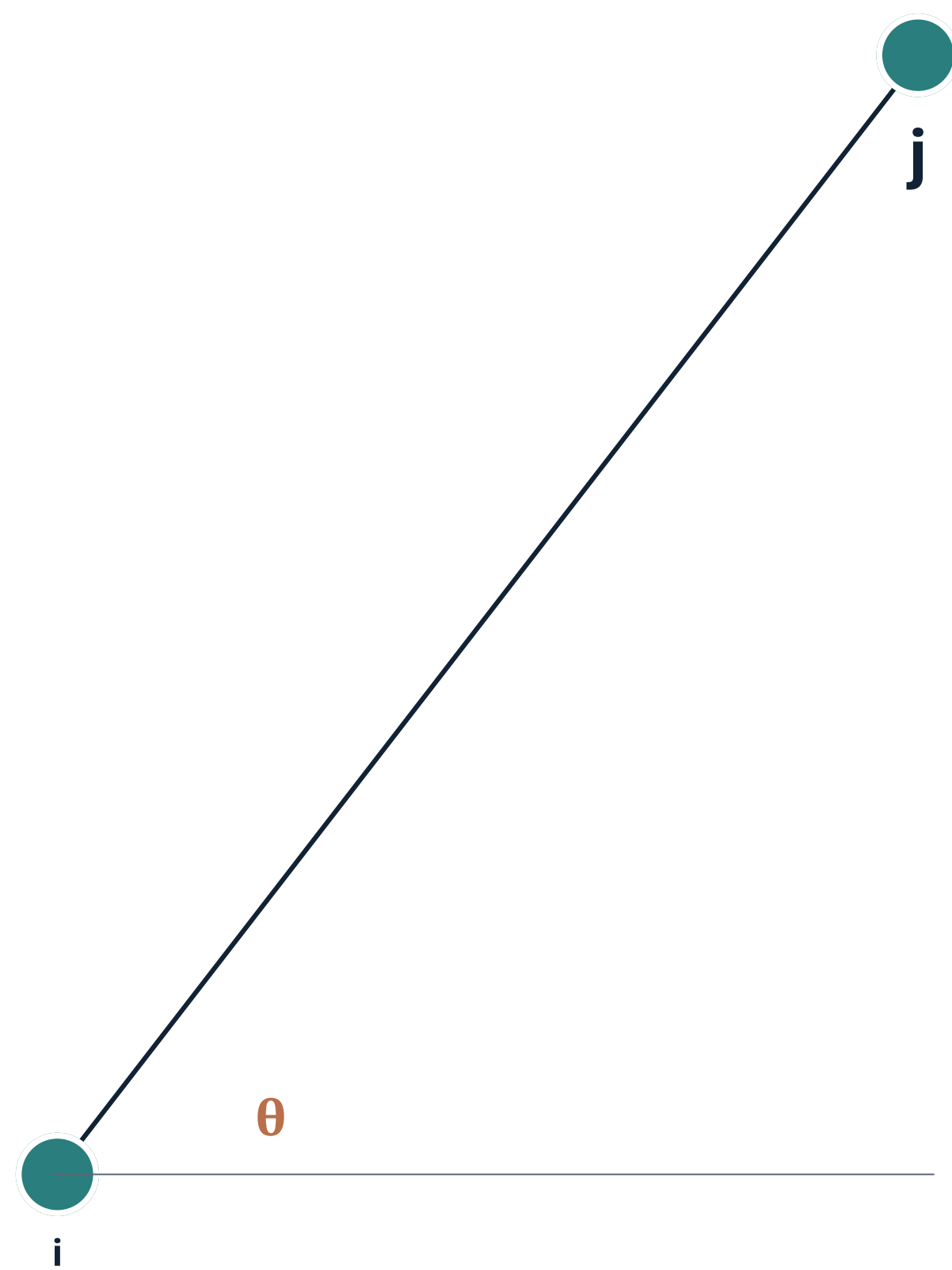
ENGINEERING MEANING

The matrix converts member-end deformation into member-end resisting actions in the member's local coordinate system.

Local → Global Transformation

The transformation matrix is what makes the solver applicable to horizontal, vertical and inclined frame members.

MEMBER GEOMETRY



$$L = \sqrt{(\Delta x^2 + \Delta y^2)}$$

$$\theta = \text{atan2}(\Delta y, \Delta x)$$

TRANSFORMATION MATRIX

[T], c = cosθ, s = sinθ

c	s	0	0	0	0
-s	c	0	0	0	0
0	0	1	0	0	0
0	0	0	c	s	0
0	0	0	-s	c	0
0	0	0	0	0	1

$$[K]_e = [T]^T [k]_e [T]$$

```
T = transformation(theta)
```

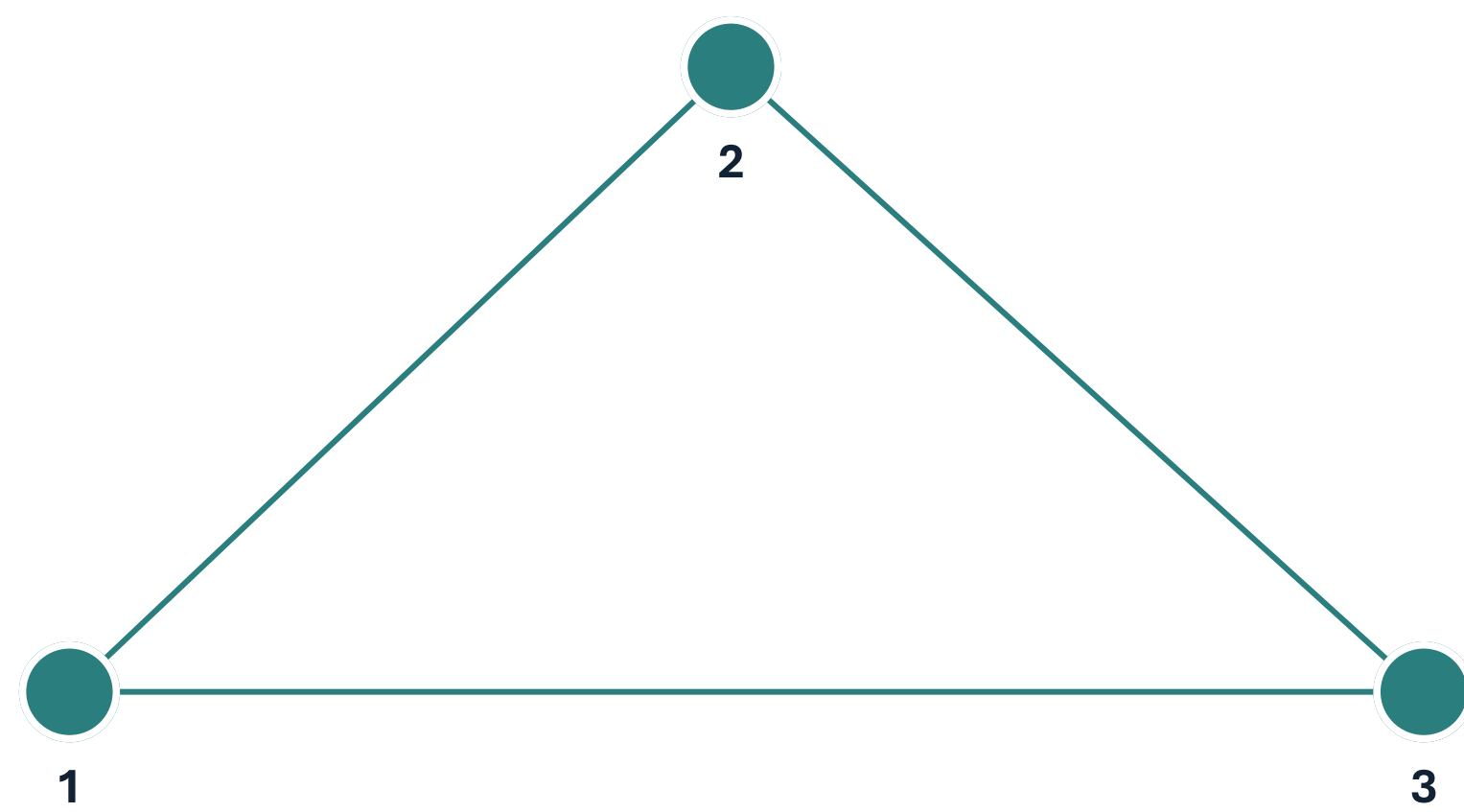
```
k_global = T.T @ k_local @ T
```

One local formulation + one transformation = a generic solver for arbitrary member orientation.

Building the Global Stiffness Matrix

The finite-element assembly operation maps every member's six local DOFs into the correct global matrix locations.

ASSEMBLY VISUAL



For member i–j:

- 1. Build k_{local}
- 2. Transform to k_{global}
- 3. Map six member DOFs
- 4. Add into K_{global}

GLOBAL [K]
assembled

IMPLEMENTED AS A DIRECT MATRIX ASSEMBLY LOOP

```
dof_map = [get_dof(i,0), get_dof(i,1), get_dof(i,2), get_dof(j,0), get_dof(j,1), get_dof(j,2)]
```

```
K_global[dof_map[a], dof_map[b]] += k_global[a,b]
```

This nested accumulation is the computational heart of the Direct Stiffness Method.

LOAD VECTOR

$\{F\}$ = nodal loads – fixed-end forces

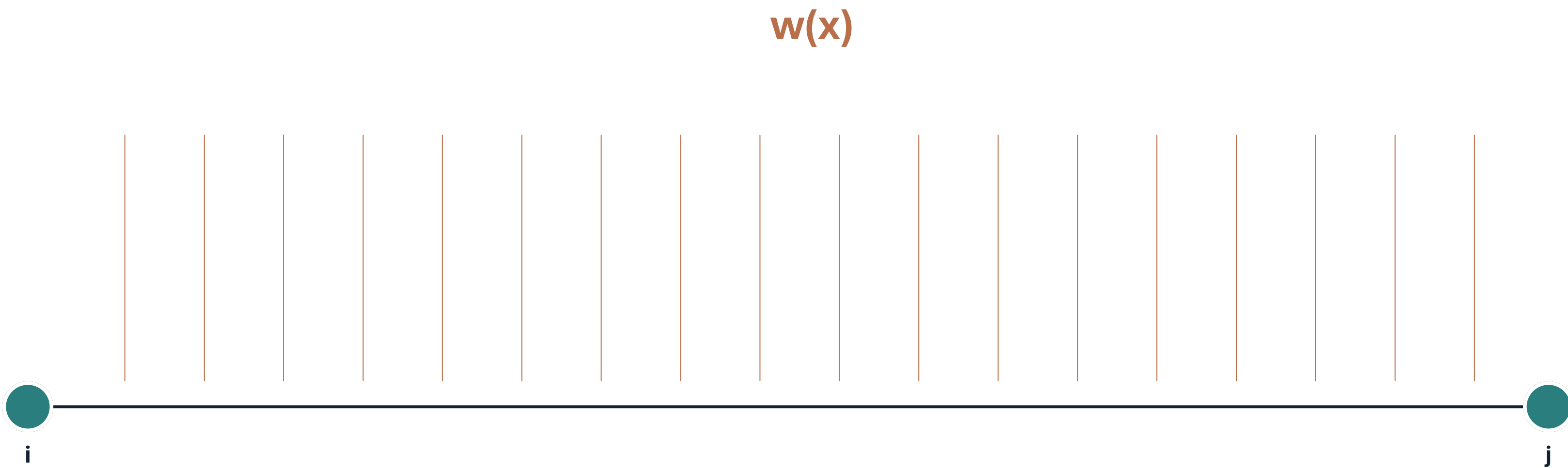
GLOBAL EQUILIBRIUM

$$[K]\{U\} = \{F\}$$

From Distributed Load to Fixed-End Forces

Member loading is converted into equivalent fixed-end forces so it can participate in the global nodal equilibrium system.

LOAD ON A MEMBER



The load is first represented in the member's local system.

INPUT

```
Member | w1 | w2 | w3
```

w1 and w2 define distributed-load end intensities.
If w3 is non-zero, the three-point UVL routine is used.
A 6×1 local fixed-end-force vector is returned.

IMPLEMENTATION

```
_member_fef(member_id, L)
```

```
fef_global = T.T @ fef
```

The transformed vector is then subtracted from the global load vector during assembly.

Turning Supports into a Solvable System

Support conditions are translated into free and fixed DOF sets before the reduced linear system is solved.

PARTITIONED EQUATIONS

$$[K_{uu} \ K_{uk}] \begin{Bmatrix} U_u \end{Bmatrix} = \{F_u\}$$

$$[K_{ku} \ K_{kk}] \begin{Bmatrix} U_k \end{Bmatrix} = \{F_k\}$$

$$\{U_u\} = [K_{uu}]^{-1} (\{F_u\} - [K_{uk}]\{U_k\})$$

CODE-LEVEL PARTITIONING

```
if n[key] == 1: free_dofs.append(dof) else: fixed_dofs.append(dof)
```

```
U_u = np.linalg.solve(K_uu, F_u - K_uk @ U_known)
```

FREE DOFs

Unknown nodal translations and rotations are obtained from the reduced system.

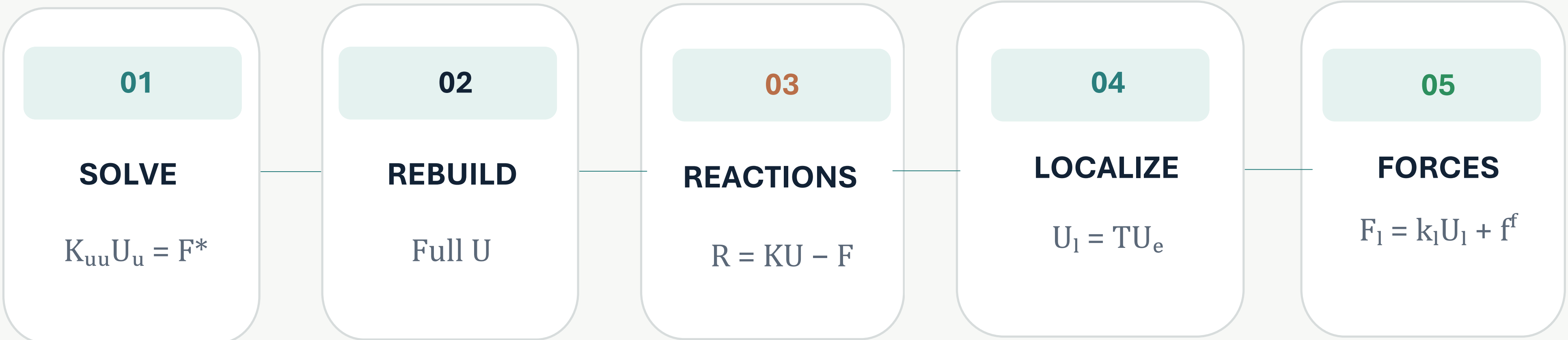
FIXED DOFs

Prescribed support values are retained and coupled into the reduced equations.

If the reduced matrix is singular, the solver reports an unstable or inadequately restrained structure.

From the Displacement Vector to Engineering Results

The analysis does not stop at U: it reconstructs the full response, reactions and local member-end forces.



SUPPORT REACTIONS

$R = K_{\text{global}} U - F_{\text{global}}$

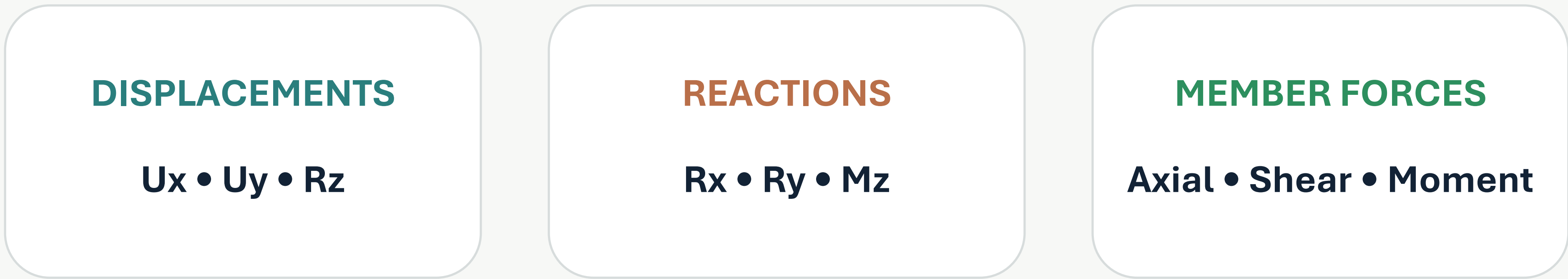
$R = K_{\text{global}} @ U - F_{\text{global}}$

MEMBER END FORCES

$F_l = k_l U_l + f^f$

$F_{\text{local}} = k_{\text{local}} @ U_{\text{local}} + f_{\text{ef}}$

RESULT PACKAGE



This creates a clean separation between the numerical solution and engineering interpretation.

Global U → reactions at restraints → local member forces

Excel Is the Model Definition Layer

Three required sheets keep the input compact, structured and easy to edit.

NODES

Node • X • Y • H • V • M • ux_free • uy_free • rz_free • ux_val • uy_val • rz_val

MEMBERS

Member • Node_i • Node_j • E • A • I

LOADS

Member • w1 • w2 • w3

VALIDATION BEFORE ANALYSIS

Required sheets: Nodes, Members and Loads.

Required columns are checked before analysis.

Missing coordinates or E/A/I properties are rejected.

Member connectivity must reference existing nodes.

Zero-length members are rejected.

If w3 is absent, the code creates it as zero for safe load handling.

The spreadsheet is therefore a controlled structural-model interface, not just a data dump.

How the Code Is Organized

The project separates structural mechanics from the interface and output layers.

USER
INTERFACE

app.py

CustomTkinter window • workflow • status
• result tables

NUMERICAL
ENGINE

fem_solver.py

StructuralSolver • stiffness assembly •
solve • recovery

UI
COMPONENTS

ui_components.py

Cards • tables • badges • theme

DATA / EXPORT

exporters.py +
Pandas/OpenPyXL

Excel • CSV • text report

SAMPLE MODEL

sample_model.py

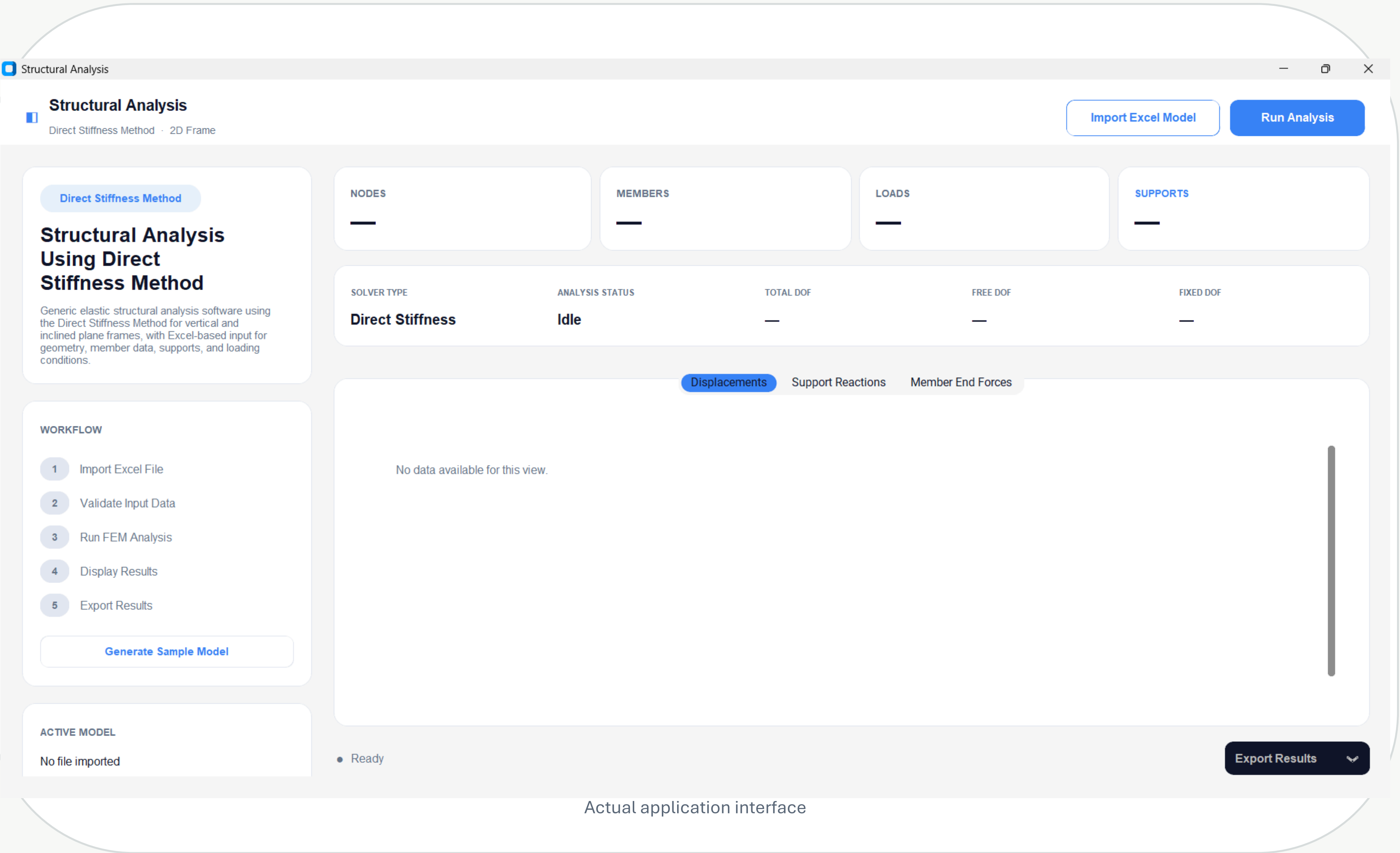
Generates a sample frame workbook for
demonstration

DESIGN PRINCIPLE

A GUI change should not require rewriting the structural mechanics. A solver change
should not require rebuilding the entire interface.

A Workflow-Oriented Structural Analysis GUI

The interface follows the same sequence as the underlying computational method.



IMPLEMENTATION DETAIL

The analysis runs in a worker thread while the GUI updates progress, status, DOF counts and result tables after completion.

Representative Analysis Results

The sample run demonstrates that the application reaches a completed solution and exposes multiple result views.

8
NODES

8
MEMBERS

3
LOADS

3
SUPPORTS

SYSTEM SIZE

24 DOF

FREE

16

FIXED

8

Structural Analysis

Direct Stiffness Method - 2D Frame

Import Excel Model

Run Analysis

Direct Stiffness Method

Structural Analysis Using Direct Stiffness Method

Generic elastic structural analysis software using the Direct Stiffness Method for vertical and inclined plane frames, with Excel-based input for geometry, member data, supports, and loading conditions.

NODES

MEMBERS

LOADS

SUPPORTS

8

8

3

3

SOLVER TYPE

ANALYSIS STATUS

TOTAL DOF

FREE DOF

FIXED DOF

Direct Stiffness

Completed

24

16

8

Displacements

Support Reactions

Member End Forces

Node

Rx (kN)

Ry (kN)

Mz (kN-m)

1.000

-72.764

-35.885

403.182

4.000

-73.993

207.819

407.308

8.000

-19.243

123.066

-0.000

WORKFLOW

1 Import Excel File

2 Validate Input Data

3 Run FEM Analysis

4 Display Results

5 Export Results

Generate Sample Model

ACTIVE MODEL

sample_double_frame.xlsx

Analysis complete

Export Results

Support reactions

Structural Analysis

Direct Stiffness Method - 2D Frame

Import Excel Model

Run Analysis

Direct Stiffness Method

Structural Analysis Using Direct Stiffness Method

Generic elastic structural analysis software using the Direct Stiffness Method for vertical and inclined plane frames, with Excel-based input for geometry, member data, supports, and loading conditions.

NODES

MEMBERS

LOADS

SUPPORTS

8

8

3

3

SOLVER TYPE

ANALYSIS STATUS

TOTAL DOF

FREE DOF

FIXED DOF

Direct Stiffness

Completed

24

16

8

Displacements

Support Reactions

Member End Forces

Member

Axial_L (kN)

Shear_L (kN)

Moment_L (kN-m)

Axial_R (kN)

Shear_R (kN)

Moment_R (kN-m)

1.0000

-35.8850

72.7639

403.1817

35.8850

-72.7639

324.4570

2.0000

-14.7276

28.9256

-92.8132

14.7276

77.1404

-35.9853

3.0000

77.1404

-14.7276

35.9853

-77.1404

14.7276

-140.1252

4.0000

207.8190

73.9935

332.6265

-207.8190

-73.9935

407.3083

5.0000

3.6604

105.1407

35.0997

-3.6604

105.9914

-41.6430

WORKFLOW

1 Import Excel File

2 Validate Input Data

3 Run FEM Analysis

4 Display Results

5 Export Results

Generate Sample Model

ACTIVE MODEL

sample_double_frame.xlsx

Analysis complete

Export Results

Member end forces

REPRESENTATIVE OUTPUT

Node 1: Rx = -72.764 kN | Ry = -35.885 kN | Mz = 403.182 kN·m

The result tables are populated automatically after a successful analysis.

Nodal Displacements: What the Solver Actually Solves

The displacement table exposes each node's three DOFs and whether each one was free or fixed.

Structural Analysis

Structural Analysis

Direct Stiffness Method · 2D Frame

Import Excel Model

Run Analysis

Direct Stiffness Method

Structural Analysis Using Direct Stiffness Method

Generic elastic structural analysis software using the Direct Stiffness Method for vertical and inclined plane frames, with Excel-based input for geometry, member data, supports, and loading conditions.

WORKFLOW

1

Import Excel File

2

Validate Input Data

3

Run FEM Analysis

4

Display Results

5

Export Results

Generate Sample Model

ACTIVE MODEL

sample_double_frame.xlsx

NODES

8

MEMBERS

8

LOADS

3

SUPPORTS

3

SOLVER TYPE

Direct Stiffness

ANALYSIS STATUS

Completed

TOTAL DOF

24

FREE DOF

16

FIXED DOF

8

Displacements

Support Reactions

Member End Forces

Node	DOF	Type	Displacement (mm)
1	Ux	Fixed	0.000
1	Uy	Fixed	0.000
1	Rz	Fixed	0.000
2	Ux	Free	293.845
2	Uy	Free	0.044
2	Rz	Fixed	0.000

Analysis complete

Export Results

FREE DOF

Unknown displacement is obtained from the reduced stiffness system.

FIXED DOF

Prescribed displacement remains known and is inserted into the full vector.

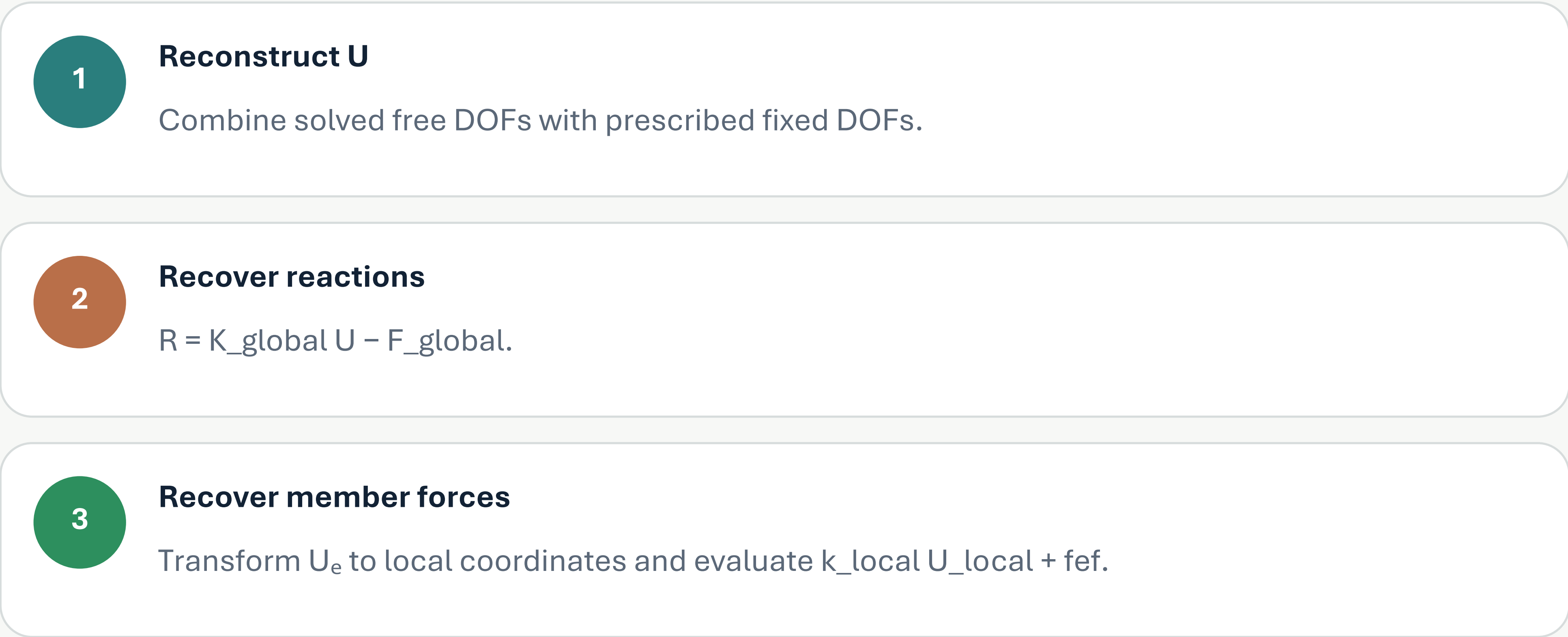
Example visible output: Node 2 Ux ≈ 293.845 mm and Uy ≈ 0.044 mm.

What Each Result Means

The application preserves the distinction between kinematic response, support actions and member actions.



POST-PROCESSING LOGIC



RESULT TRACEABILITY

Every displayed quantity can be traced back to the global displacement solution and the same element-level equations used during assembly.

Validation & Numerical Safeguards

The implementation checks the model before solving and surfaces important numerical failure modes.

	Workbook structure	Nodes, Members and Loads sheets are required.
	Input completeness	Missing X/Y or E/A/I values are rejected.
	Connectivity	Member end nodes must exist.
	Geometry	Zero-length members are rejected.
	Boundary conditions	At least one free DOF is required.
	Numerical stability	Singular reduced stiffness matrices are reported as instability / inadequate restraint.

NEXT VALIDATION STEP

Benchmark representative frames against hand calculations and trusted structural-analysis software, then add automated regression tests.

What the Project Achieves — and What Comes Next

A technically credible report should make both the capabilities and the boundaries explicit.

CURRENT CAPABILITIES

2D plane-frame elastic analysis.
3 DOFs per node.
Inclined-member transformation.
Excel-based model definition.
Member distributed-load representation.
Support reactions.
Member end forces.
Interactive GUI + export.

FUTURE EXTENSIONS

Automated benchmark suite.
Deflected-shape visualization.
Shear-force and moment diagrams.
More explicit point-load positions.
Sparse matrix solution for larger models.
Nonlinear analysis as a separate layer.
Formal verification and regression testing.

ENGINEERING BOUNDARY

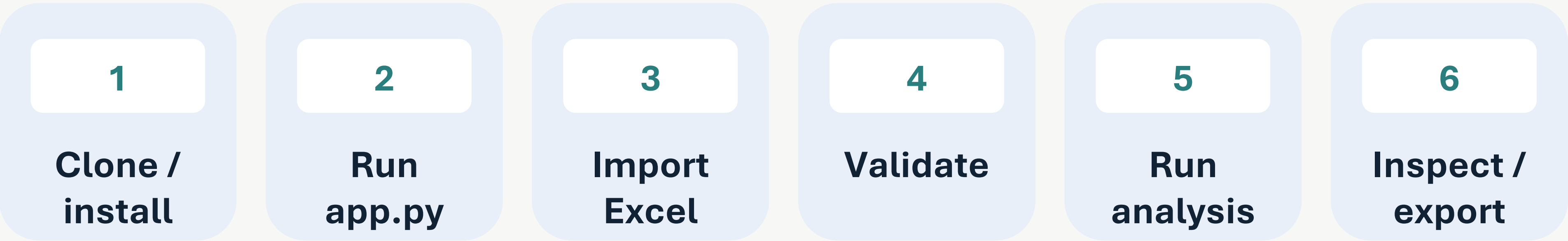
The current work is a computational/educational analysis framework. Verification and validation are the necessary next step before production engineering use.

Technology Stack & Run Sequence

The project uses familiar scientific-Python tools and a clear execution sequence.

Python	Core implementation
NumPy	Matrix operations + linear solve
Pandas	Tabular input / output
OpenPyXL	Excel workbook handling
CustomTkinter	Desktop GUI

RUN SEQUENCE



Repository: github.com/sejal-010605/Computational-Structural-Analysis-Using-Direct-Stiffness-Method

The actual Excel workbook behind the solver

The submitted workbook is not illustrative: it is the structured model definition used for the computational analysis.

NODES — GEOMETRY + CONSTRAINTS + NODAL LOADING

1	0	0	0	0	0	0	0	0	0	0	0
2	0	5	0	0	0	1	1	1	0	0	0
3	3	9	0	0	0	1	1	1	0	0	0
4	5	7	0	0	0	1	1	1	0	0	0
5	9	11	0	0	0	1	1	1	0	0	0
6	12	6	-10000	0	0	1	1	1	0	0	0
7	12	0	0	0	0	0	0	0	0	0	0

MEMBERS — CONNECTIVITY + SECTION/MATERIAL PROPERTIES

3	3	4	0.01	2000000000000	8.330e-06
4	4	5	0.01	2000000000000	8.330e-06
5	5	6	0.04	2000000000000	1.330e-04
6	6	7	0.04	2000000000000	1.330e-04

LOADS — DISTRIBUTED MEMBER LOADING

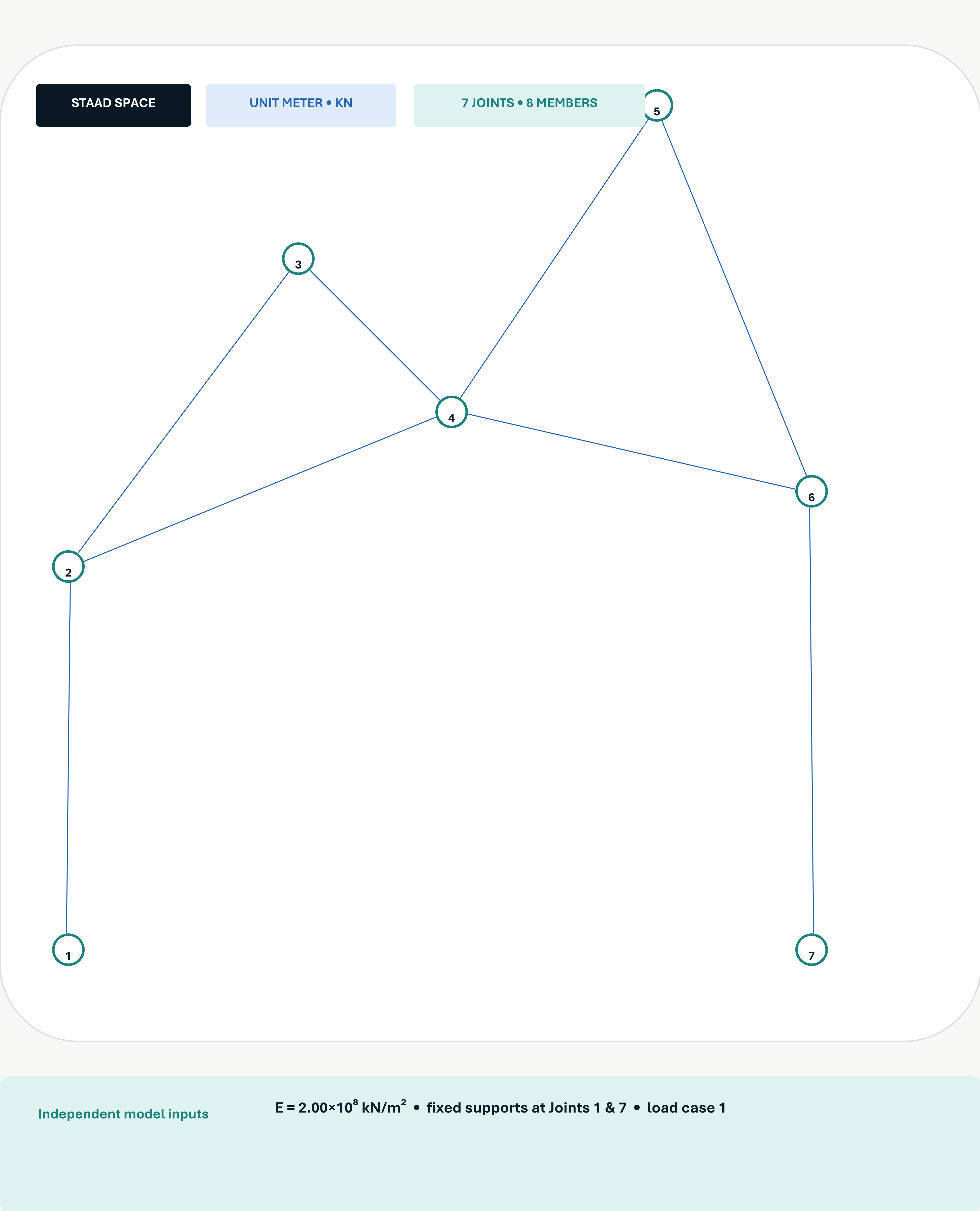
5	UDL	LOCAL	-10000	-10000	0
4	TRI	LOCAL	-10000	-5000	0
2	TRI	LOCAL	-2000	-5000	0

THREE-SHEET DATA MODEL

Nodes • Members • Loads

The same frame is explicitly defined in STAAD.Pro

The supplied .STD file provides an independent structural model for verification of the Python implementation.



What is actually encoded in the STAAD input file

The verification model contains explicit coordinates, incidences, section properties, materials, supports and the applied load case.

```
JOINT COORDINATES
1 0 0 0; 2 0 5 0; 3 3 9 0
4 5 7 0; 5 9 11 0; 6 12 6 0; 7 12 0 0

MEMBER INCIDENCES
1 1 2; 2 2 3; 3 3 4; 4 4 5
5 5 6; 6 6 7; 7 2 4; 8 4 6

SUPPORTS
1 7 FIXED

MEMBER LOAD
2 TRAP Y -2 -5
4 TRAP Y -10 -5
5 UNI Y -10
```

VERIFICATION LOGIC

Geometry

Coordinates and incidences reproduce the irregular frame topology.

Supports

The file explicitly restrains both end joints.

Loading

Equivalent distributed-load definitions are present in the independent model.

INDEPENDENT REFERENCE MODEL

Python results versus STAAD.Pro

The reported comparison from the submitted analysis shows close agreement across support reactions and moments.

Response	Python	STAAD.Pro	Abs. diff	Rel. diff
V reaction A (kN)	28.890	28.802	0.088	0.30%
H reaction A (kN)	18.421	18.357	0.064	0.35%
M _a (kN·m)	-41.082	-40.923	0.159	0.39%
V reaction B (kN)	41.610	41.698	0.088	0.21%
H reaction B (kN)	-2.421	-2.357	0.064	2.64%
M _β (kN·m)	-41.573	-42.790	1.217	2.93%

ENGINEERING READING

The comparison supports the numerical implementation — while also highlighting where modelling conventions matter.

Strong agreement

Most listed quantities differ by less than 0.5%.

Largest listed difference

Joint B moment: approximately 2.93%.

Interpretation

Agreement is meaningful when geometry, properties, supports, loads and sign conventions are equivalent.

Thank you